

이 력 서

○ 인적 사항

	성 명	강 민 건	지원분야	웹 개발자(풀스택 및 Devops)
	생년월일	1994 년 11 월 5 일 (만 31세)	희망연봉	회사 내규에 따름
	휴 대 폰	010 - 9824 - 1421	비상시(관계)	010 - 7574 - 1421 (아버지)
	이 메 일	toub094@naver.com		
	주 소	서울특별시 관악구 관악로 186, 1505호		

○ 학력 사항

재학 기간	학교명	학위	전공	소재지	구분
2018.03 ~ 2019.02	한국영상대학교	전문학사	실용음악과 보컬전공	세종시	졸업
2013.03 ~ 2014.02	서울예술전문학교	학점은행제	실용음악과 보컬전공	서대문구	수료
2010.03 ~ 2012.02	충북고등학교		이과	청주시	졸업

○ 경력 및 사회 활동 / 수상 내용

근무 기간	연수	회사명	소재지	부서/직위	업무 내용
2019.01 ~ 2020.01	1	한국영상대학교 대학평의위원회	세종시	의원	교내 학칙 및 학과 개정 방안 심의
2019.02 ~ 2020.02	1	한국영상대학교 등록금심의위원회	세종시	위원	교내 대학 등록금 심의
2019.02 ~ 2020.02	1	한국영상대학교 총학생회	세종시	총학생회장	대학 총학생회 사업 총괄
2019.04 ~ 2021.04	2	세종특별자치시 청년정책과	세종시	위원	세종시 청년 예술 문화 사업 추진 관련 정책 예산 심의
2020.03 ~ 2020.09	6개월	이단옆차기 실용음악학원	인천/수원	실장	강사 및 수강생 스케줄 관리 및 상담 안내
2023.01 ~ 2023.05	5개월	동아방송 실용음악학원	원주시	강사	수강생 보컬 트레이닝 교육
2023.01 ~ 2023.05	5개월	동아크누아	원주시	사원	음반 제작 및 유통, 홈페이지 제작 및 운영
2023.05 ~ 2023.11	6개월	쿠팡풀필먼트서비스 유한회사 곤지암 1센터	경기 광주시	사원	출고 공정 및 WMS 기반 출고 문제해결
2024.03 ~ 2024.10	7개월	쿠팡풀필먼트서비스 유한회사 이천 3센터	경기 이천시	사원	재고 관리 및 상품 검품
2025.01 ~ 2025.07	6개월	쿠팡풀필먼트서비스 유한회사 시흥 1센터	경기 시흥시	사원	출고 공정 및 WMS 기반 출고 문제해결
2025.09 ~ 2025.12	3개월	(주)우아한형제들 B마트	신림동	사원	주문 상품 입고 및 포장

○ 교육 사항

교육 기간	교육 과정	교육 기관
2026.03 ~ 2026.09	ChatGPT를 활용한 웹 개발자 및 DevOps 과정	MBC교육아카데미학원 천호점

○ 자격 및 면허 사항

취득년월일	자격명 / 면허명	발령처
2019.02.26	자동차운전면허 1종 보통	충북지방경찰청
2020.11.06	한국사능력검정시험 자격증 4급	국사편찬위원회

○ 병력 사항

병역필	군별	육군	계급	병장	복무기간	2015.12.15 ~ 2017.09.14
-----	----	----	----	----	------	-------------------------

위에 기재한 사항은 사실과 다름없음을 확인합니다.
2026년 7 월 21 일 성 명 : 강 민 건 (인)

자 기 소 개 서

지 원 자	강 민 건	연 락 처	010-9824-1421
성 장 과 정 및 학 창 시 절			
[매일의 기록이 만든 25%의 변화] 저는 문제를 감이 아니라 기록으로 해결하는 습관을 어머니의 옷 가게에서 배웠습니다. 학창 시절 옷 가게를 운영하시던 어머니를 도우며 입고 상품 확인, 진열, 재고 관리, 고객 응대까지 가게 일 전반을 익혔습니다. 그런데 일을 하다 보면 분명 창고에 있어야 할 옷이 안 보이거나, 진열해 둔 옷에서 뒤늦게 하자가 발견되어 손님 앞에서 난처해지는 일이 자주 생겼습니다. 처음에는 기억을 더듬어 물건을 찾아다녔는데 그럴수록 같은 실수만 반복됐습니다. 고민 끝에 매장에 진열된 옷과 창고에 쌓아둔 옷을 품목별로 리스트에 적어두고 매일 마감 전에 실물과 대조하기 시작했습니다. 본사에서 상품이 들어오면 상태부터 확인해서 판매 가능한 것과 아닌 것을 나누는 기준도 정했습니다. 번거로운 작업이었지만 이게 쌓이니 재고 누락이 확실히 줄었고, 항상 좋은 상태의 옷을 내놓게 되면서 월 평균 매출이 약 25% 오르는 결과까지 이어졌습니다. 사람의 기억은 틀려도 기록은 오류를 잡아낸다는 걸 이때 몸으로 배웠습니다. 나중에 테스트와 코드 리뷰로 안정성을 지키는 개발 문화를 접했을 때 바로 공감할 수 있었던 것도 이 경험 덕분이라고 생각합니다. 대학에서는 총학생회장을 맡아 4,500여 명의 학우를 대표했습니다. 학식 불만이 커졌을 때는 민원을 그대로 전달하는 대신 전체 재학생 설문을 직접 만들어 원인을 항목별로 분석했고, 이를 근거로 한 개선안이 학교 운영위원회에 반영되어 학식 품질이 실제로 나아졌습니다. 축제 때는 기획 및 운영 총괄로서 예산안과 정산안을 심의했는데, 학우들의 회비가 쓰이는 일이라 항목마다 근거를 따졌고 정산 때는 영수증과 지출 내역을 일일이 대조했습니다. 축제 포스터 기획과 디자인도 맡으면서 보는 사람 눈에 어떤 정보가 먼저 들어와야 할지 고민했던 기억이 있는데, 사용자에게 보이는 결과물을 만드는 재미를 처음 느낀 순간이었습니다. 10명의 임원과 부서별로 역할을 나누고 매주 회의하면서 혼자 하는 것보다 나누고 소통할 때 결과가 좋아진다는 것도 배웠습니다. 문제를 기록으로 정의하고 절차로 검증하며 팀과 함께 완성하는 습관. 이것을 풀스택 개발과 DevOps 직무에서 그대로 이어가고 싶습니다.			
성 격 의 장 단 점			
[왜 그렇게 동작하는지부터 묻는 나] 저는 일을 맡으면 그 업무의 본질과 원리부터 파고드는 성격입니다. CFS에서 OB 출고 업무를 했을 때도 피킹, 패킹, 리젝, OPS 등 배정되는 업무를 단순히 시키는 대로 하는 데 그치지 않고, 각 공정이 왜 이런 순서로 설계되어 있는지, 어느 지점에서 오류가 생기기 쉬운지를 먼저 이해하고 나서 일을 시작했습니다. 반품 상품이 들어오면 검품하고 재고에 반영하는 업무를 맡았을 때도 마찬가지로 했습니다. 상품 상태만 확인하고 넘기는 대신 어떤 이유로 반품이 발생했는지, 재판매가 가능한 상태인지를 하나하나 따져 분류했습니다. 그렇게 파고들다 보니 반품 검품이 단순 확인 작업이 아니라 재고 정확도와 다음 판매의 품질까지 좌우하는 일이라는 걸 이해하게 됐고, 업무를 대하는 기준도 달라졌습니다. 원리를 알고 나니 처음 맡는 업무에도 빠르게 적응할 수 있었고, 전산 데이터의 흐름만 보고도 앞으로 생길 문제를 예측해 미리 대비할 수 있었습니다. 기술이 빠르게 바뀌			

는 개발 분야에서도 오래 가는 힘은 결국 동작 원리에 대한 이해라고 생각합니다. 프레임워크가 바뀌어도 원리를 아는 사람은 흔들리지 않는다고 믿기에, 이 성격이 문제의 근본 원인을 찾아야 하는 개발 업무와 가장 잘 맞는 저의 강점이라고 생각합니다.

또 하나의 강점은 조직을 움직여 본 경험에서 나온 소통 방식입니다. 총학생회장으로서 10명의 임원과 4,500여 명의 학우 사이에서 일할 때, 저는 지시보다 경청이 조직을 움직인다는 것을 배웠습니다. 매주 회의에서 각 부서의 어려움을 먼저 듣고 역할을 조정했고, 학우들의 요구는 짐작하는 대신 설문으로 직접 확인한 뒤 정책에 반영했습니다. 방향은 리더가 제시하되 결정의 근거는 모두에게 투명하게 공유하는 것, 그래야 사람들이 납득하고 움직인다는 것이 제가 배운 리더십이자 소통의 방식입니다.

다만 단점도 분명합니다. 목표와 논리 중심으로 소통하다 보니, 한때 동료들과의 감정적인 교류를 뒤로 미룬 채 일 얘기만 한 적이 있습니다. 업무는 정확하게 굴러갔지만 팀의 분위기가 가라앉으면 동기부여와 성과도 함께 떨어진다는 걸 뒤늦게 깨달았습니다. 그 후로는 논리를 꺼내기 전에 상대의 상황과 감정부터 묻는 연습을 하고 있습니다. 코드 리뷰처럼 서로의 결과물에 의견을 주고받는 것이 일상인 개발 문화에서, 원리에 대한 깊이와 사람에 대한 공감을 함께 갖춘 동료로 일하고 싶습니다.

경 력 사 항

[관리자 없는 마감, 그러나 침착하게 문제 상황을 신속한 대응으로 증명한 책임]

CFS 곤지암 1센터에서 관리자 없이 혼자 상온 챔버 마감 출고를 책임졌던 날이 있습니다. 당시 관리자 수가 부족한 상황이라 PS사원인 제가 상온 챔버를 배정받았고, 혼자 PS 전산 장비를 챙겨 OB 데스크에서 MOS를 보며 멀티와 싱글 집품의 잔여 유닛 수부터 파악했습니다. 남은 물량과 시간을 계산한 뒤 배정받은 인원 두 분을 각 구역에 나누어 배치하고, 원바코드 권한을 부여해 집품을 진행했습니다.

문제는 마감에 임박한 시점에 터졌습니다. 전산 서버 오류로 집품을 해도 유닛 수가 줄어들지 않았고, 관리자님들도 상황 파악을 위해 분주하게 움직이고 있었습니다. 솔직히 당황할 만한 상황이었지만, 지금 이 챔버를 책임지는 사람은 나라고 생각하며 당장 할 수 있는 일부터 순서를 정했습니다. 서버 복구를 기다리는 동안 마감에 직결되는 싱글 토트부터 찾아 관리자님 지시에 따라 상온 출고를 진행했고, 문제없이 허브까지 직접 전달했습니다. 서버가 정상화되자마자 사원분들께 현재 상황을 설명하고, 구역별로 남은 집품 개수를 다시 파악해 할당을 재분배했습니다. 이후 플로우 관리자님께 상황을 보고해 조치를 받고, 남은 재집건까지 WMS와 PMS로 처리하면서 11시 30분과 50분, 두 번의 마감을 모두 놓치지 않고 업무를 마쳤습니다.

이날 배운 것은 분명합니다. 장애 상황에서 중요한 것은 복구를 기다리는 것이 아니라, 지금 가능한 일의 우선순위를 정하고 관련자들에게 상황을 공유하는 것이었습니다. 전산이 멈춰도 현장은 멈출 수 없었고, 그 간극을 메우는 건 결국 사람의 판단이었습니다. 시스템 장애가 현장에 어떤 영향을 주는지 사용자 입장에서 직접 겪어본 경험은, 장애에 강한 서비스를 만들고 문제 상황에서 침착하게 대응해야 하는 풀스택·DevOps 개발자에게 흔치 않은 자산이라고 생각합니다. 이 경험을 개발 현장에서 그대로 이어가겠습니다.

지 원 동 기 및 입 사 후 포 부

[시스템의 사용자에서, 시스템을 만드는 사람으로]

로켓배송과 새벽배송을 현장에서 직접 움직여 본 경험이 저를 개발자의 길로 이끌었습니다. 국내 E-Commerce의 정점에 있는 CFS 출고 현장에서 일하면서, 수천 명의 작업을 정확하게 굴러가게 만드는 것은 결국 잘 설계된 시스템이라는 걸 매일 체감했습니다. 스캔 한 번의 응답 속도, 화면에 뜨는 정보의 순서 하나가 현장의 실수와 마감을 좌우했고, 마감 직전 서버 장애가 현장을 어떻게 흔드는지도 직접 겪었습니다. 처음에는 불편함으로 다가왔던 것들이 어느 순간부터는 궁금증으로 바뀌었습니다. 이 화면은 왜 이렇게

설계됐을까, 어떻게 하면 더 나아질 수 있을까. 그 질문들이 쌓이면서 빅데이터와 자동화로 진화하는 이 산업에서 시스템을 쓰는 사람이 아니라 만들고 개선하는 사람이 되고 싶다는 목표가 생겼고, 그것이 웹개발자 직무에 지원한 이유입니다.

입사 후 제가 기여할 수 있는 가장 큰 자산은 사용자를 추측하지 않아도 된다는 점입니다. 어떤 화면이 현장에서 왜 불편한지, 어떤 개선이 실제 생산성으로 이어지는지 몸으로 알고 있습니다. 특히 시스템이 멈췄을 때의 대가를 현장에서 치러본 사람으로서, 서비스는 새로 만드는 것 못지않게 안정적으로 유지보수하는 일이 중요하다는 것을 누구보다 잘 압니다. 화려한 신규 기능보다 장애 없이 돌아가는 하루가 현장에는 더 큰 가치라는 걸 알기에, 유지보수와 운영 안정성을 개발의 기본기로 여기는 개발자가 되겠습니다.

저는 회사를 거쳐 가는 곳이 아니라 함께 성장하는 곳으로 생각합니다. 입사 초기에는 현장과 개발 조직 사이의 언어를 통역하며 팀에 빠르게 녹아들고, 이후에는 프론트엔드부터 백엔드, 배포와 운영, 유지보수까지 서비스의 전체 수명주기를 책임질 수 있는 개발자로 성장해 회사가 새로운 도전을 할 때 믿고 맡길 수 있는 사람이 되겠습니다. 현장에서 보낸 시간이 저를 개발자의 길로 이끌었듯, 이제는 제가 만드는 시스템이 어느 현장에서 일하는 누군가의 하루를 조금 더 수월하게 만들어 주기를 바랍니다. 그 마음으로 회사의 성장 단계마다 제 역할을 넓혀가며, 오래 곁에서 함께 성장하는 개발자가 되겠습니다.

○ 프로젝트

프로젝트 명 (주제)	집다움(JIPDAUM) 1차 — 홈퍼니싱 D2C 이커머스 쇼 핑몰 백엔드 구축 및 인증 시스템 구현	작업 기간	26.05.28~06.11
주요역할 및 담당	Django 기반 백엔드 전반을 담당했습니다. 프로젝트를 기능별 6개 앱(Users, Products, Orders, Payments, core 등)으로 설계하고, Oracle DB·CORS·DRF 환경을 구축했습니다. 9개 모델의 설계와 마이그레이션을 진행했으며, 초기에 통합했던 models.py를 조인 최적화를 위해 도메인별로 재분할하는 구조 개선을 주도했습니다. JWT 기반 회원가입·로그인·로그아웃 API와 이메일 인증코드 방식의 비밀번호 재설정을 구현하고, django-allauth와 dj-rest-auth로 카카오·네이버·구글 소셜 로그인을 연동했습니다. API 키를 .env로 분리해 관리하고, .gitignore와 requirements.txt를 정비해 팀 GitHub 협업 환경을 구축했습니다.		
사용 스킬	Python, Django, Django REST Framework, Oracle DB, JWT(simplejwt), django-allauth, dj-rest-auth, Git/GitHub		
성과 및 느낀점	이 단계에서 가장 기억에 남는 건 모델 구조를 두 번이나 갈아엎은 일입니다. 처음에는 앱별로 나뉜 models.py가 관리하기 복잡해서 하나로 합쳤는데, 막상 DB 연동과 조인을 최적화하려니 다시 도메인별로 나누는 게 맞았습니다. 결국 같은 코드를 세 번 만졌습니다. 그때는 시간 낭비처럼 느껴졌지만, 지나고 보니 설계라는 게 처음부터 완성되는 게 아니라 데이터 관계를 따라가며 계속 고쳐야 하는 것임을 배운 과정이었습니다. 중간에는 서비스가 통째로 멈춘 적도 있습니다. User 모델을 별도 앱으로 분리해 놓고 AUTH_USER_MODEL 경로를 안 고쳐서 LookupError가 났는데, 원인을 찾고 보니 설정 파일의 참조 경로 한 줄이었었습니다. 고치는 건 금방이었지만, 설정 한 줄이 시스템 전체를 세울 수 있다는 걸 그때 처음 실감했습니다. 소셜 로그인도 카카오, 네이버, 구글 세 곳을 연동했는데 provider마다 앱 등록 절차부터 콜백 처리까지 다 달라서 각 개발자 센터 문서를 붙잡고 하나씩 확인하며 완성했습니다. 이 과정에서 발급받은 API 키를 코드에 두면 안 되겠다 싶어 .env로 분리하고 .gitignore를 정비했는데, 지금 생각하면 보안을 습관으로 만들기 시작한 지점이었습니다. 회원가입 쪽은 아이디와 이메일 중복, 비밀번호 규칙을 serializer 단계에서 검증하게 만들어 잘못된 데이터가 DB까지 가지 않도록 했고, requirements.txt를 정리해 팀원들이 같은 환경을 바로 만들 수 있게 했습니다. 백엔드를 처음부터 끝까지 세워보니, 개발은 코드를 쓰는 시간보다 구조와 설정을 바로잡는 시간이 절반이었습니다.		
프로젝트 명 (주제)	집다움(JIPDAUM) 2차 — Spring Boot 기반 결제 시스템 구현 및 보안 강화 (풀스택)	작업 기간	26.07.10~07.30
주요역할 및 담당	백엔드를 Spring Boot로 전환하며 Cart, Order, Coupon, Product, Review 도메인 전체를 설계하고 PortOne V2 연동으로 결제 준비·검증 API를 구현했습니다. 결제 검증 우회 차단, 사용자 API 접근 제한, OAuth 토큰 전달 방식 개선, JWT 타입 검증, 동시성 제어 등 보안 취약점을 전면 수정했습니다. 기존 Django 백엔드의 로컬 인증 전체를 Spring으로 이관하고, DB를 Oracle에서 MySQL로 전환하며 두 백엔드가 같은 사용자 테이블을 공유하는 통합 구조를 완성했습니다. 프론트엔드에서는 인증 플로우 모달화, 다크모드, 한국관 페이지를 구현했습니다.		
사용 스킬	Java, Spring Boot, JPA/Hibernate, JWT, OAuth2, PortOne 결제 API, MySQL, Oracle, React, Tailwind CSS, Git		

성과 및 느낀점	PortOne 결제 연동을 처음 끝냈을 때는 솔직히 뿌듯했습니다. 그런데 코드를 다시 뜯어보면서 생각이 달라졌습니다. 결제 검증을 안 거치고도 주문이 완료되는 우회 엔드포인트가 있었고, 인증 없이 열려 있는 사용자 API도 나왔습니다. 하나씩 정리했습니다. 우회 경로는 제거해 모든 결제가 서버 검증을 통과하게 일원화했고, 소셜 로그인 토큰을 URL에 담아 보내던 방식은 히스토리와 로그에 남는 위험을 확인하고 1회용 코드 교환으로 바꿨습니다. 쿠폰 중복 사용과 재고 초과 차감 문제는 파고들어 보니 조회 후 수정 방식 자체의 한계여서, 조건부 원자적 UPDATE로 DB 수준에서 막았습니다. 후반에는 두 백엔드를 하나로 묶는 통합을 맡았습니다. Oracle에서 MySQL로 전환하는데 분명 존재하는 테이블을 Spring이 못 찾는 오류가 났고, 한참 헤맨 끝에 Hibernate 네이밍 전략이 테이블명을 소문자로 바꿔 Django가 만든 대문자 테이블과 어긋난다는 원인을 찾았습니다. 네이밍 전략을 고치고 사용자 조회도 공유 테이블을 보게 수정해, 어느 쪽으로 가입해도 같은 계정으로 동작하는 구조를 만들었습니다. SYSDATE 같은 Oracle 전용 문법도 표준으로 걷어냈습니다. 이 단계를 지나며 두 가지가 남았습니다. 돌아가는 코드와 안전한 코드는 전혀 다르다는 것, 그리고 시스템 통합의 어려움은 코드 안쪽이 아니라 시스템과 시스템이 만나는 경계에서 생긴다는 것입니다.
----------	--

프로젝트 명 (주제)	집다움(JIPDAUM) 3차 — 디버깅 및 유지보수: 테스트 체계와 운영 환경 정비	작업 기간	26.08.18~09.10
주요역할 및 담당	서비스 안정화를 위한 디버깅과 유지보수 체계 구축을 담당했습니다. 빌드 실패와 환경 의존적 오류를 추적·해결하고, Map 기반 요청·응답을 record DTO(@Valid)로 전환해 오류가 런타임 전에 걸러지는 구조를 만들었습니다. 쿠폰·JWT·주문 서비스에 단위 테스트를 추가하고, Docker·Docker Compose로 실행 환경을 컨테이너화했으며, 설정 예시 파일과 인수인계 문서를 정리해 어떤 환경에서도 프로젝트를 재현할 수 있게 했습니다.		
사용 스킬	MySQL, Docker, Docker Compose, Spring Boot, Hibernate, Django, 환경변수 관리, Git		
성과 및 느낀점	기능이 갖춰진 뒤에 시작한 이 단계에서 저의 일은 만드는 것보다 지키는 것이었습니다. 처음 맞닥뜨린 건 빌드 자체가 안 되는 문제였습니다. 원인을 추적해 보니 pom.xml에 섞여 들어간 문자 하나가 파싱 오류를 일으키고 있었고, 그걸 걷어내고서야 빌드가 살아났습니다. Windows에서만 나던 충돌도 있었습니다. 대소문자만 다른 domain.User와 domain.user 패키지가 공존했는데, 대소문자를 구분하지 않는 Windows 파일시스템에서 실제 충돌로 이어진 것이라 패키지 구조를 정리해 해결했습니다. 이런 오류들을 겪으며 문제를 하나 고치는 것보다 다시 안 생기게 만드는 게 유지보수라는 생각이 들었고, 구조를 손보기 시작했습니다. Map으로 주고받던 요청과 응답을 전부 record DTO와 @Valid 검증으로 바꿔 잘못된 데이터가 로직에 닿기 전에 걸러지게 했고, 쿠폰, JWT, 주문 서비스에 단위 테스트를 붙였습니다. 이후로는 코드를 고칠 때마다 테스트가 먼저 실수를 잡아줬는데, 그 안정감을 한번 경험하니 테스트 없이 수정하던 때로 돌아갈 수 없었습니다. 마지막으로 Docker와 Docker Compose로 실행 환경을 컨테이너화하고 설정 예시 파일과 인수인계 문서를 정리했습니다. 문서를 쓰다 보니 오히려 제가 구조를 다시 이해하게 됐고, 팀원이 제 문서만 보고 환경을 그대로 세우는 걸 봤을 때는 기능이 돌아갈 때와는 다른 보람이 있었습니다. 유지보수는 코드가 아니라 재현성과 문서에서 시작된다는 것, 그것이 이 단계가 남긴 가장 큰 경험이자 배움이었던 것 같습니다.		